Dynamic Memory Management

Computer Programming 1

Outline



- Dynamic allocation and deallocation of the memory
- Arrays and Strings allocated dynamically
- Multi-dimensional arrays allocated dynamically

The dynamic use of the memory

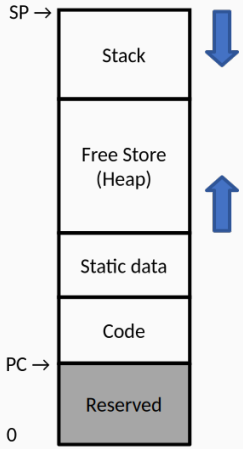


- Static allocation forces you to define the structure and size of the data in advance (compile time)
  - It is not always acceptable and/or convenient
  - Example: the size of an array fixed and established a priori (‘int a[100];‘)
- It is also possible to manage the memory dynamically, that is, during the execution of the program

Model of memory management for a program

Memory area allocated to an execution of a program:

- Code (program area): intended to contain the instructions (in machine language)
- Static data area: intended to contain global or statically allocated variables and constants of the program
- Heap area: intended to hold dynamic variables (of unpredictable dimensions at compilation time)
- Stack area: intended to contain the local variables and formal parameters of functions



Dynamic Memory



- Code and Static data are fixed: content of a cell will change (when you assign variables, for instance), but the number of cells if fixed at compilation time
- Heap:
  - Part is free part contains data created dynamically
  - The access to this area is via pointers
  - Allocation and deallocation are done by means of `new` and `delete`
- Stack
  - Contains frames
  - A frame keeps track of current call to a function and contains function arguments, local variables, return address, state
  - If a function calls a function, you add a frame
  - LIFO: last in, first out

The operators new and new[]



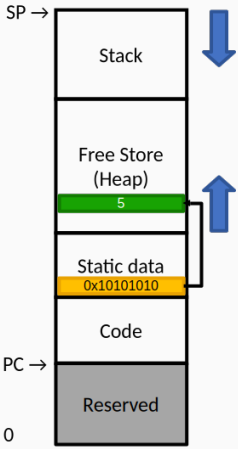

```
new type
new type (value) // initialize with value
new type [dim]   // allocate N-elements
```

```
int *p, *q;
char *string;
p = new int;
q = new int (5); // Initialize memory with 5
string = new char [3 * i];
```

The operators new and new[]

1. new allocates a memory area suitable for containing an object (or objects) of the specified type
2. If it is specified, it initializes it to value
3. Returns the address of the first element of the area
4. Typically assigned to a pointer

```
int * a;         // "a" is in static data
a = new int(5); // new is in heap and "a" points to heap
```



The operators delete / delete[]




```
delete pointer
delete[] pointer // for arrays
```

- The value of the expression `address` must be the address of a memory cell previously allocated by the call of a `new` instruction

```
int *p, *q;
char *string;
p = new int;
string = new char[30];
delete p;
delete[] string;
```

The operators delete / delete[]



- The `delete/delete[]` operators deallocate the previously allocated memory area starting from the specified address
  - If the address does not correspond to a call to `new` an error is raised
  - An area allocated by `new` must be deallocated with `delete`
- At the end of the program, the memory allocated with `new` is also automatically deallocated

  Note
  - *Memory deallocation means: area is no longer "reserved" (it can be reused)*
  - *Memory deallocation different from "content is deleted":*
    - *Data still accessible (security risk, if sensitive)*
    - *Easy not to notice an error situation*

Mixing new and delete



- Mixing `new/delete[]` and `new[]/delete` is a bad idea and in general its behavior is undefined
- In fact:
  - `new[]/delete` will deallocate only the first element, leaving memory corrupted
  - `new/delete[]`: When allocating with `[]`, C++ stores the size of the allocated memory area somewhere. When `delete[]` is called, this value is read, to free memory. However, if not set, a random value will be read and a random area of the heap will be freed

Tip: Use `-Wall` when compiling

Duration of the dynamic allocation



- A dynamically created object remains allocated until:
  - It is explicitly deallocated with the `delete` operator
  - The program ends
- The memory allocated with `new`, and not explicitly deallocated with `delete`, is no longer available for other programs: waste of memory (memory leak) and degradation of machine performances
- General rule: In a program, you must always explicitly deallocate everything you have allocated dynamically as soon as it is no longer needed.

Dynamic management of memory: Pros and Cons



- Pros:
  - Efficient memory management: it allocates only the necessary space
  - It allows the creation of dynamic data structures (lists, trees, ...)
- Cons:
  - Much more difficult to manage
  - Easy to introduce errors and/or memory leaks
- Note
  - There are tools to support the identification of memory leaks: open-source tools, e.g., valgrind, gperftool, -fsanitize=...
  - Commercial tools, e.g., Parasoft Insure++, IBM Rational Purify

Dynamic allocation of strings



- Dynamic allocation of strings allows you to create strings of different sizes at run-time
- A dynamic string is a pointer to the first element of the character sequence
- You need to ensure the string is properly terminated with '\0'
- The I/O of a dynamic string is automatically managed by the operators », «
- All primitives on strings in <cstring> also apply to dynamic strings

```
char * sc, *sb = new char [20];
cin >> sb;
sc = new char[strlen(sb)+1];
strcpy(sc, sb);
cout << sc;
```

Failure of new



- The execution of new may not be successful (no memory for the program) in this case the C++ standard provides that, unless otherwise specified, the new requires to the OS to abort the program
- Solution: use ”new (nothrow)”
  - With the ”nothrow” option, new does not abort but returns ”NULL” in case of inability to allocate the requested memory

```
char *p = new (nothrow) char[mymax];
...
if (p!=NULL)...
```



Remember that:

```
int a[10];          // array of 10 elements
int *b = a;         // array decay to pointer to its first row
int (*b)[10] = &a;  // reference keeps type
a[1] == *(a + 1)

int a[10][20];          // 2D
int (*b)[20] = a;       // decays to a pointer to its first row
int (*c)[10][20] = &a; // pointer to array [10][20]
a[1][3] == *(*(b + 1) + 3) == (*c)[1][3]
```

Dynamic Arrays (1)




```
int *a = new int[10]; // array of 10 integers
a                     // pointer to integer (first element of the array)
&a                    // pointer to full array: (int *)[10]
a[3] == *(a + 3)      // fourth element
delete[] a;           // delete the full array
```

2D Arrays



- 2D arrays are more complex
- Three implementations:
  - as a 1D array, simulating rows and columns with pointers math
  - as a 2D array
  - as an array of pointers (the pointers point to the columns)

As a 1D array




```
int row_size = 2;
int col_size = 3;

// Allocate a single contiguous block
int* arr = new int[row_size * col_size];

// C++ does not know how much cols we have per
// row. We do the math
arr[row * col_size + col] = 42;

delete[] arr;
```

```
|<----------  2 * 3 --------------->|
+-----------------------------------+
| 0,0 | 0,1 | 0,2 | 1,1 | 1,2 | 1,3 |
+-----------------------------------+
```

Dynamic Arrays (2)



- Suppose we want C++ to do the math for us:

```
???? a = new int[10][20]; // array of 10 integers
```

- The type is a pointer to the first element, that is, the first element is an array of `[20]` integers

```
int (*a)[20] = new int[10][20];
a                   // pointer to int[20]
*a                  // address of first row
(a + 2)             // address of third row
a[1][3] == *((*a + 1) + 3)   // fourth element of first row
delete[] a
```

Dynamic Arrays: Jagged array (3)




```
int*[10]                // array of 10 pointers to integers
int **a = new int*[10]; // pointer to array of 10 pointers to integers
a[0] == *(a + 0)        // pointer to integer (first element)
a[0] = new int;         // new int -> int*
a[1] = new int[10];     // new int[10] -> int* (to array of ten elements)
a[1][2] == *((*a + 1) + 2)
```

Dynamic Arrays: Jagged array (4)



./cpp/jagged_array.cpp

Recap



- Contiguous memory is usually preferred
- When using an array of pointers, we need to deallocate columns first and then the array of pointers, to avoid memory leaks

Allocation	Type	Access	Deallocation	Contiguous
new int[R*C]	int *	*(a + r*C + c)	delete[]	Yes
new int[R][C]	(int *)[C]	a[r][c]	delete[]	Yes
new int*[C]	int **	a[r][c]	delete[r], delete[]	No

Can a function return an Array?



- A function can return an array if it is dynamically allocated inside the function itself
- Size is still an issue (it has to be passed along)

```
int *times(int a[], ...) {
  int * b = new int[10]; // pointer to an array dynamically
                         // allocated inside the functions
  // ...
  return b; // the function returns the address to
            // the first element of the dynamic array
}

int v[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
int * w = times(v,...); // the function call
```

Remarks



- When using dynamic allocation (e.g., of an array) that is passed between more than a function, the programmer must:
  - decide which function is responsible for allocating the data
    - lack of allocation → segmentation fault
    - multiple allocations → memory leak
  - decide which function is responsible for deallocating it
    - lack of deallocation → memory leak
    - multiple deallocations → segmentation fault
- Consider the case of a function deallocating arguments, which gets called twice

  Note. It is essential to agree in advance the responsibility for the allocation e deallocation when the code is developed in a team!